exec.pyc

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Worked for 50s >

Este arquivo é malware.

Análise passo a passo

1. **Tipo de arquivo**

   É um `.pyc` (bytecode Python compilado). O magic number `f30d0d0a` indica Python 3.11+. O cabeçalho tem 16 bytes e o restante é um objeto `code` serializado via `marshal`.

2. **Obfuscação em múltiplas camadas**
   - Nomes de variáveis e funções totalmente aleatórios (`aqgqzxkfjzbdnhz`, `idzextbcjbgkdih`, `fvzsrnzlh`, etc.).
   - O código principal importa `base64` e `zlib`, faz `b64decode` + `decompress` de uma string enorme embutida e depois usa `exec` / `compile`.
   - A string embutida, após `base64` + `zlib`, ainda está criptografada com XOR usando a chave constante `134`.
   - Só depois do XOR o código-fonte legível aparece.

3. **Comportamento do código desofuscado**

   O payload final (salvo em `/home/workdir/artifacts/deobfuscated.py`) faz o seguinte:
   - Aguarda 10 segundos.
   - Verifica se o sistema é russo (locale, variáveis de ambiente `LANG` / `LANGUAGE`, timezone, offset UTC). Se detectar Rússia, **aborta a execução** (comportamento clássico de malware russo para não infectar máquinas locais).
   - Consulta a blockchain Solana (vários endpoints públicos de RPC) no endereço fixo `BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8SC` buscando memos de transações. O memo contém um JSON com um campo `link` (URL codificada em base64).
   - Baixa o conteúdo dessa URL e extrai dos cabeçalhos HTTP (nomes ofuscados em base64: `ivbase64` e `secretkey`) uma chave secreta e um IV.
   - Baixa um Node.js portátil oficial (versão 22.9.0) de acordo com o SO/arquitetura.
   - Cria um arquivo temporário `i.js` contendo:

```JavaScript
eval(atob('...payload...'));
```

Copied link to clipboard. Shared links can be viewed by anyone with the link.

junto com a `secretKey` e o IV.

- Executa o Node.js de forma silenciosa (`stdout` / `stderr` redirecionados para `/dev/null` , janela oculta no Windows).

4. **Indicadores de malícia**
   - Packing agressivo + XOR + nomes aleatórios.
   - Uso da blockchain Solana como canal de comando e controle / dead-drop (técnica moderna e difícil de bloquear).
   - Download e execução de segundo estágio via `eval` em Node.js.
   - Anti-análise geográfica (pula execução em sistemas russos).
   - Persistência/controle de reexecução via arquivo `~/init.json` (válido por 2 dias).
   - Execução stealth (sem janela, sem saída padrão).

Conclusão final

Malware (dropper/downloader de estágio múltiplo que usa Solana como C2 e entrega um payload Node.js ofuscado).

Não é um arquivo benigno sob nenhuma circunstância.

Qual é a categoria principal deste malware?

Worked for 6s >

Categoria principal: Downloader (Loader / Dropper)

Classificação detalhada

Este arquivo (o `.pyc`) é um **Downloader** (também chamado de Loader ou Dropper de múltiplos estágios).

Ele não realiza o roubo de dados ou ações destrutivas por si só. Sua função principal é:

1. Verificar se o sistema é russo (e abortar se for).
2. Consultar a blockchain Solana (endereço `BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8SC`) para obter a URL do próximo estágio via memo de transação.
3. Baixar o Node.js portátil.
4. Baixar um payload JavaScript criptografado (AES).
5. Executá-lo silenciosamente via `eval(atob(...))` .

Contexto da família

Este sample faz parte da campanha **GlassWo**
compromete repositórios Python no GitHub v

Copied link to clipboard. Shared links can be viewed by anyone with the link.

ser um **Infostealer** (roubo de carteiras de criptomoedas, tokens NPM/GitHub, chaves SSH, credenciais de navegadores etc.).

Resumo da cadeia:

- Este arquivo → **Downloader/Loader**
- Payload final → **Infostealer** (principalmente focado em desenvolvedores e crypto)

Portanto, a categoria principal **deste arquivo específico** é **Downloader**.

↘ Qual a categoria de um infostealer?

↘ Como ele obtém a URL do payload?

↘ Quais são os riscos de download em Python?

Copied link to clipboard. Shared links can be viewed by anyone with the link.